RAG, LangChain & Qdrant Interview Knowledge

Production-minded retrieval-augmented generation reference

Purpose
RAG architecture, chunking, embeddings, Qdrant retrieval, LangChain composition, citations, evaluation and failure diagnosis.

Audience: Final-year engineering students and fresher candidates targeting Python, software development, ML, AI engineering and GenAI roles.

RAG design: Content is explicit and retrieval-friendly so chunks can stand alone. Source identifiers, definitions, examples, troubleshooting and interview reasoning are intentionally separated.

Table of Contents

1. 1. What RAG is
2. 2. Complete RAG architecture
3. 3. Chunking and metadata
4. 4. Embeddings and similarity
5. 5. Qdrant collections and retrieval
6. 6. LangChain components and versioning
7. 7. Grounded prompts and citations
8. 8. RAG versus fine-tuning versus plain LLM
9. 9. Evaluation and troubleshooting

1. What RAG is

Retrieval-augmented generation combines information retrieval with generation. Instead of relying only on model parameters, the application retrieves relevant external content and supplies it as context.

RAG is useful for private/current documents and source citations. It does not guarantee correctness because retrieval itself can fail or retrieve contradictory material.

Approach	Knowledge source	Main strength	Main weakness
Plain LLM	Model + prompt	Simple	May lack private/current facts
RAG	Retrieved documents	Grounding/citations	Retrieval failure
Fine-tuning	Model weights	Behavior/domain adaptation	Not a live document lookup

2. Complete RAG architecture

Ingestion: load → clean → split → metadata → embed → upsert. Query: question → embed/rewrite → retrieve → optional filter/rerank → context construction → grounded generation → citations.

Keep ingestion repeatable and idempotent. Query-time code should not re-index the corpus. A stable chunk identifier makes duplicate detection and source citation much easier.

```
Question
  -> query embedding
  -> Qdrant top-k retrieval + filters
  -> context with source IDs
  -> grounded Gemini/LLM prompt
  -> answer + source references
```

3. Chunking and metadata

Large chunks can dilute relevance and consume generation context; tiny chunks can lose necessary context. Overlap can preserve continuity but also increases duplication. There is no universal best chunk size. Measure on representative questions.

Useful metadata includes source filename, document title, section/page, chunk ID, document version and topic. Metadata can support filtering and citations.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120)
chunks = splitter.split_documents(documents)
for i, chunk in enumerate(chunks):
    chunk.metadata["chunk_id"] = f"{chunk.metadata.get('source','doc')}:{i}"
```

4. Embeddings and similarity

A document embedding and a query embedding should come from a compatible embedding model so they occupy the same vector space. The model determines vector dimensions and affects similarity behavior.

Qdrant supports distance metrics including cosine, dot product, Euclidean and Manhattan. Select a metric consistent with the embedding model and validate retrieval empirically.

5. Qdrant collections and retrieval

Qdrant stores points: a vector plus optional payload. A collection is a named set of points with compatible vector configuration. Payload can contain source text and metadata.

Current Qdrant Python documentation uses `query_points` for query operations. Older tutorials/integrations may show other APIs, so verify the installed qdrant-client version before copying code.

```
from qdrant_client import QdrantClient
client = QdrantClient(url=QDRANT_URL, api_key=QDRANT_API_KEY)
result = client.query_points(
    collection_name="interview_knowledge",
```

```
    query=query_vector,
    limit=5,
    with_payload=True,
).points
for point in result:
    print(point.score, (point.payload or {}).get("source"))
```

6. LangChain components and versioning

LangChain is a compositional framework for documents, splitters, embeddings, vector stores, retrievers, prompts, models and output parsing. Python integrations are distributed across packages, and APIs evolve. Current projects may use packages such as `langchain-text-splitters` and `langchain-qdrant`.

Do not claim a single chain API is universal. Verify imports and method signatures against installed versions. For interviews, explain the architecture and data contracts first.

```
# Illustrative current-style integration; verify versions before use.
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_qdrant import QdrantVectorStore

splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120)
chunks = splitter.split_documents(documents)
vector_store = QdrantVectorStore.from_documents(
    chunks, embedding=embeddings,
    url=QDRANT_URL, collection_name="interview_knowledge"
)
```

7. Grounded prompts and citations

The prompt should define an evidence boundary: use retrieved context when relevant, say when evidence is insufficient, and do not invent sources.

Citations should be derived from retrieved metadata. A model can format a citation, but the application should be the authority for which chunks were actually retrieved.

```
SYSTEM:
Answer using CONTEXT when it supports the question.
If evidence is insufficient, say so.
Do not invent source names or page numbers.

CONTEXT:
[chunk_id=python-042 | source=01_Python_Interview_Knowledge.pdf]
...
```

8. RAG versus fine-tuning versus plain LLM

RAG changes inference-time context; fine-tuning changes model parameters. Frequently changing factual documents are commonly better handled by retrieval because they can be re-indexed without retraining.

A plain LLM call is appropriate when no external/private/current evidence is needed. Adding RAG is justified when the retrieval benefit exceeds its operational complexity.

9. Evaluation and troubleshooting

Evaluate retrieval and generation separately. Retrieval evaluation asks whether useful evidence appears in top-k. Generation evaluation asks whether the answer is correct, grounded, complete and properly cited.

Common failures: poor chunk boundaries, inconsistent embedding model, duplicate indexing, stale vectors, irrelevant top-k results, missing filters, incorrect vector dimensions and source metadata errors.

Symptom	Likely cause	First check
No relevant source	Ingestion/embedding issue	Inspect raw points
Relevant chunk ranked low	Chunk/query mismatch	Review chunk boundaries and queries
Duplicate results	Repeated ingestion	Stable IDs and collection counts

Symptom	Likely cause	First check
Wrong citation	Model invented source	Render from metadata
Old answer	Stale vectors	Document version/re-index state
Too much context	top-k too large	Calibrate retrieval/threshold

Scenario: duplicate indexing

Situation: The ingestion job runs twice and generates new IDs for the same chunks.

Reasoning: Use deterministic chunk IDs or document/version IDs and upsert rather than blindly insert. Inspect collection counts before and after ingestion.

Scenario: one PDF is never retrieved

Situation: Check that it was ingested, embedding dimensions/model match the collection, filters do not exclude it, and the query has semantic overlap. Run a direct Qdrant query and inspect payloads before changing the LLM prompt.

Reasoning: Changing generation prompts while retrieval is broken.

Scenario: retrieval finds nothing useful

Situation: If top results are below a calibrated threshold, either return a controlled “insufficient evidence” response or use an explicitly labeled general-knowledge fallback if the product permits it. Do not pass irrelevant chunks as if they were evidence.

Reasoning: If top results are below a calibrated threshold, either return a controlled “insufficient evidence” response or use an explicitly labeled general-knowledge fallback if the product permits it. Do not pass irrelevant chunks as if they were evidence.

Interview question

Why use RAG instead of placing all documents in a prompt?

Strong answer: RAG selects a relevant subset, scales better with corpus size, reduces irrelevant context and can attach source metadata.

Follow-up: What happens if retrieval is wrong?

Interview question

What is chunk overlap?

Strong answer: Repeated boundary content between adjacent chunks. It can preserve context across split boundaries but increases storage and duplicate retrieval, so it should be tuned empirically.

Follow-up: How would you test chunking?

Interview question

What does top-k mean?

Strong answer: The number of highest-ranked retrieval candidates returned. Larger k can improve recall but increases irrelevant context, latency and generation load.

Follow-up: How would you choose k?

Interview question

Why must query and document embeddings be compatible?

Strong answer: Similarity search assumes the vectors share a representation space and compatible dimensions/metric behavior. Mixing unrelated embedding models makes similarity scores unreliable.

Follow-up: What happens when the embedding model changes?

Interview question

RAG or fine-tuning for changing company documents?

Strong answer: Usually RAG because documents can be updated and re-indexed without retraining. Fine-tuning can still be useful for behavior/style, but is not a live document store.

Follow-up: Can fine-tuning make factual updates easier?

Interview question

What is a relevance threshold?

Strong answer: A cutoff used to reject weak retrieval matches. It must be calibrated because similarity scores depend on the embedding model, metric and corpus.

Follow-up: Why is a universal threshold unsafe?

Interview question

How do you evaluate a RAG system?

Strong answer: Create representative questions with known supporting chunks, measure retrieval recall/rank, then separately evaluate answer correctness, groundedness, completeness and citation accuracy.

Follow-up: Why separate retrieval and generation?

Revision checklist

- Explain ingestion and query pipelines.
- Explain chunking, overlap and metadata.
- Explain embeddings and similarity.
- Explain Qdrant points/collections/payload.
- Know current `query_points` pattern and version caveat.
- Explain LangChain package/version dependence.
- Generate citations from retrieved metadata.
- Make ingestion idempotent.
- Handle stale/duplicate vectors.
- Evaluate retrieval separately from generation.

Reference notes

Primary documentation used for version-sensitive guidance. Verify exact package/API behavior against the versions installed in your project.

https://docs.langchain.com/oss/python/langchain/overview

https://docs.langchain.com/oss/python/integrations/vectorstores/qdrant

https://qdrant.tech/documentation/search/search/

https://qdrant.tech/documentation/manage-data/collections/

https://qdrant.tech/documentation/concepts/points/

End of knowledge base document